

**NATIONAL UNIVERSITY OF MODERN
LANGUAGE ISLAMABAD**



Semester project

Simple Voting System

Submitted to:

Ms. Fatima Gillani

Group members:

Group members:	
Name	Roll no
Zulqarnain hassan	SP21379
Ehsan Mehmood	SP21371

Department: BSSE

Semester: 5th

Shift: Afternoon

Date: 05-10-2024

First Deliverable Documentation: Simple Voting System.

1. Project Overview

Main Goal of the Project:

The main goal of the project is to develop a Simple Voting System with a Graphical User Interface (GUI) that allows users to vote for their preferred candidates in a transparent, real-time environment. The system is designed to provide a user-friendly platform where users can cast their votes easily and view the updated vote counts instantly.

Problem the Software Aims to Solve:

In many voting scenarios (e.g., informal polls or small-scale elections), manual vote counting is time-consuming and error-prone. This project aims to solve the issue of manual vote tallying by automating the process, ensuring accuracy, transparency, and real-time feedback for the users. By providing an easy-to-use voting system, it eliminates the need for physical tallying and enhances the efficiency of the voting process.

2. Scope and Objectives

Scope:

The scope of the project includes the development of a standalone voting application with the following components:

- A user interface that allows users to select from a predefined list of candidates and cast their votes.
- Real-time vote display functionality that shows the number of votes each candidate has received.
- Admin functionality to reset the voting system or manage candidates (optional).
- The ability to save and load voting data for future use (optional).
- Simple validation to prevent multiple voting attempts by a single user (optional).
- The project will focus on providing a functional, user-friendly GUI.

Objectives:

- Develop a functional GUI-based voting system using Java.
 - Ensure that votes displayed in real-time.
 - Provide validation to ensure that users only vote once.
-

3. Technical Requirements

Technologies, Programming Languages, and Databases:

- **Programming Language: Java**
 - **Reason:** Java is platform-independent, robust, and provides rich libraries for GUI development through Swing and JavaFX. It is also widely used for building desktop applications.
- **GUI Toolkit: Swing (or JavaFX)**
 - **Reason:** Swing is a lightweight, easy-to-use framework for building desktop GUI applications in Java. It allows for the creation of forms, buttons, labels, and other UI components that will be used in the project.
- **Database: SQLite/MySQL**
 - **Reason:** SQLite provides a simple, file-based database that is easy to integrate with Java. MySQL can be used if a more complex database is required. This is optional based on whether the data persistence feature is implemented.

Why These Technologies Were Chosen:

- Java provides an object-oriented, secure, and platform-independent environment, making it suitable for desktop applications.
 - Swing offers a simple way to build GUIs with limited overhead.
 - SQLite or MySQL (if persistence is needed) are lightweight and easy to use, making them ideal for a small-scale project like this.
-

4. Functional Requirements

The functional requirements outline the core functionalities the system will offer to the users:

1. **User can vote for their preferred candidate.**
 - Users will have the ability to cast their votes by clicking on a candidate's button.
2. **Real-time vote display.**
 - The system will display the updated number of votes for each candidate in real-time as soon as a vote is cast.
3. **After voting, the user can see the vote details.**
 - Once a vote is cast, users will be able to see the total votes each candidate has received.
4. **Display a voting summary.**
 - The system will display a vote summary that shows how many votes each candidate received, optionally with percentage breakdowns.
5. **Prevent double voting.**
 - The system may include simple validation to ensure each user can only vote once.
6. **Exit the system.**
 - Users can close the application at any time, and the system will shut down gracefully.

5. Non-Functional Requirements

Non-Functional Requirements to Consider:

1. **Usability**
 - The system must provide a simple, intuitive, and user-friendly interface that does not require technical expertise to use. The voting process should be clear and easy to follow.
 2. **Performance**
 - The system must handle vote casting and display updates instantly without noticeable delay, ensuring a smooth user experience even if multiple users are voting in quick succession.
 3. **Security**
 - Basic security features like preventing duplicate voting (optional) and ensuring vote counts are handled securely must be implemented. If user authentication is implemented, it should be protected to avoid unauthorized access.
 4. **Scalability**
 - The system should be easily adaptable for adding more candidates without requiring significant changes to the underlying code.
 5. **Reliability**
 - The system must accurately track votes and ensure the correct tally is maintained throughout the voting process. In case of crashes or errors, no votes should be lost.
 6. **Maintainability**
 - The codebase should be clean, modular, and well-documented so that future developers can easily make enhancements or fix bugs.
 7. **Portability**
 - The system should run on any platform that supports Java, ensuring cross-platform compatibility (e.g., Windows, macOS, Linux).
-

Conclusion:

This document outlines the initial deliverables for the Simple Voting System project, including its scope, objectives, technical requirements, and both functional and non-functional requirements. It sets the foundation for the development process and ensures that the system meets both user and admin needs in a reliable, efficient, and user-friendly manner.